Reinforcement Learning Assignment

Instructor: Bo An, Professor, CCDS, NTU

Teaching Assistant: Penghui Yang (penghui004@e.ntu.edu.sg)

Plagiarism Policy: Your solution must be the result of your own individual effort. While you are allowed to discuss problems with your classmates, but you must not blatantly copy others' solutions.

1. Requirements

This project requires you to implement and evaluate one of the Reinforcement Learning (RL) algorithms (e.g., Q-learning, SARSA, etc.) to solve the **CliffBoxPushing** grid-world game. Novel ideas are welcome and will receive bonus credit. In this assignment, you need to implement the code on your own and present a convincing presentation to demonstrate the implemented algorithm.

The following links can help you get to know more about current RL algorithms:

- OpenAI Spinning Up: https://spinningup.openai.com/en/latest/index.html

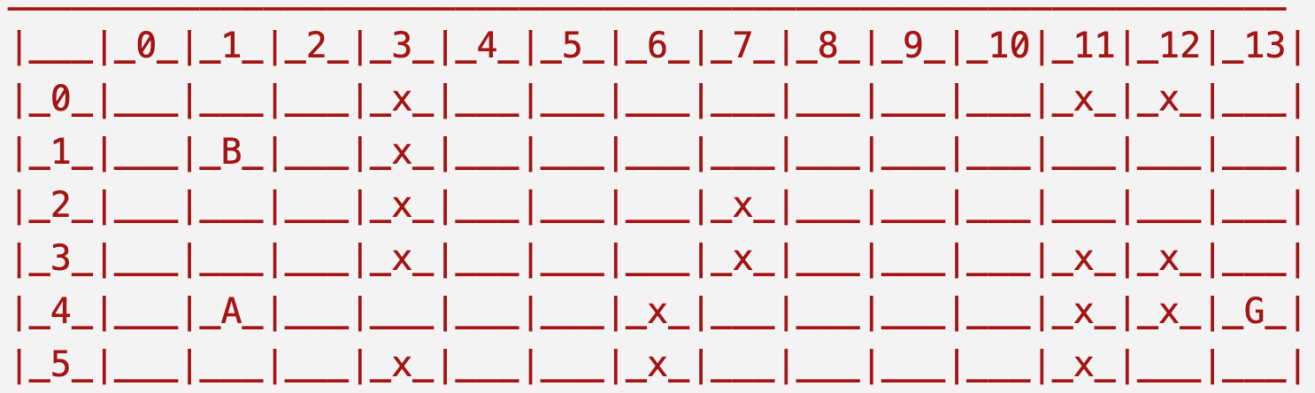


Figure 1. The Cliff Box Pushing Grid World.

2. The Environment

The environment is a 2D grid world as shown in Fig. 1. The size of the environment is 6×14. In Fig. 1, **A** indicates the agent, **B** stands for the box, **G** is the goal, and **x** means cliff. You need to write code to implement one of the RL algorithms and train the agent to push the box to the goal position. The game ends under three conditions:

1. The agent or the box steps into the dangerous region (cliff).
2. The current time step attains the maximum time step of the game.
3. The box arrives at the goal.

The MDP formulation is described as follows:

- **State:** The state consists of the position of the agent and the box. In Python, it is a `tuple`, for example, at time step 0, the state is `((4, 1), (1, 1))` where `(4, 1)` is the position of the agent and `(1, 1)` is the position of the box.
- **Action:** The action space is [1,2,3,4], which is corresponding to [`up, down, left, right`]. The agent needs to select one of them to navigate in the environment.
- **Reward:** The reward consists of
  1. the agent will receive a reward of -1 at each timestep
  2. the negative value of the distance between the box and the goal
  3. the negative value of the distance between the agent and the box
  4. the agent will receive a reward of -1000 if the agent or the box falls into the cliff.
  5. the agent will receive a reward of 1000 if the box reaches the goal position.

- **Transition:** Agent's action can change its position and the position of the box. If a collision with a boundary happens, the agent or the box would stay in the same position. The transition can be seen in the step() function of the environment.

You can check the code for further details.

3. Complete the Code

We provide the code template **RLAgent.ipynb**, which includes the implementation of the BoxPushing environment. You can visualize the grid world and manually control your agent to push the box, as well as training agent within the Jupyter Notebook (or Google Colab). We provide a random agent and an example agent class in the code for you to add your own code to train an RL agent and compare their performance. The reset() of environment initializes the positions of the agent and the box.

You need to complete the code of (marked with TODO)

1. The RLAgent class.
2. Visualization of learned policy (hint: consider all the agent position and the box position).

There is no limit on the number of iterations. You can also do reward engineering or change other part of the code, but do not modify the game setting (i.e., the map is fixed, and grading will be based on the provided map). If you do this, please clarify your modification in the report, and provide the reasons by ablation experiments (e.g., better sample efficiency).

Here is one example of visualization of learned policy.

```
[[b'_' b'_' b'_' b'x' b'_' b'_' b'_' b'_' b'_' b'_' b'_' b'x' b'x' b'_']
 [b'_' b'B' b'_' b'x' b'_' b'_' b'_' b'_' b'_' b'_' b'_' b'_' b'_' b'_']
 [b'_' b'_' b'_' b'x' b'_' b'_' b'_' b'x' b'_' b'_' b'_' b'_' b'_' b'_']
 [b'_' b'_' b'_' b'x' b'_' b'_' b'_' b'x' b'_' b'_' b'_' b'x' b'x' b'_']
 [b'_' b'A' b'_' b'_' b'_' b'_' b'x' b'_' b'_' b'_' b'_' b'x' b'x' b'G']
 [b'_' b'_' b'_' b'x' b'_' b'_' b'x' b'_' b'_' b'_' b'_' b'x' b'_' b'_']]
Please input the actions (up: 1, down: 2, left: 3, right: 4): 3
step: 1, state: (4, 0, 1, 1), actions: 3, reward: -14
Action: 3
[[b'_' b'_' b'_' b'x' b'_' b'_' b'_' b'_' b'_' b'_' b'_' b'x' b'x' b'_']
 [b'_' b'B' b'_' b'x' b'_' b'_' b'_' b'_' b'_' b'_' b'_' b'_' b'_' b'_']
 [b'_' b'_' b'_' b'x' b'_' b'_' b'_' b'x' b'_' b'_' b'_' b'_' b'_' b'_']
 [b'_' b'_' b'_' b'x' b'_' b'_' b'_' b'x' b'_' b'_' b'_' b'x' b'x' b'_']
 [b'A' b'_' b'_' b'_' b'_' b'_' b'x' b'_' b'_' b'_' b'_' b'x' b'x' b'G']
 [b'_' b'_' b'_' b'x' b'_' b'_' b'x' b'_' b'_' b'_' b'_' b'x' b'_' b'_']]
Please input the actions (up: 1, down: 2, left: 3, right: 4): 1
step: 2, state: (3, 0, 1, 1), actions: 1, reward: -15
Action: 1
[[b'_' b'_' b'_' b'x' b'_' b'_' b'_' b'_' b'_' b'_' b'_' b'x' b'x' b'_']
 [b'_' b'B' b'_' b'x' b'_' b'_' b'_' b'_' b'_' b'_' b'_' b'_' b'_' b'_']
 [b'_' b'_' b'_' b'x' b'_' b'_' b'_' b'x' b'_' b'_' b'_' b'_' b'_' b'_']
 [b'A' b'_' b'_' b'x' b'_' b'_' b'_' b'x' b'_' b'_' b'_' b'x' b'x' b'_']
 [b'_' b'_' b'_' b'_' b'_' b'_' b'x' b'_' b'_' b'_' b'_' b'x' b'x' b'G']
 [b'_' b'_' b'_' b'x' b'_' b'_' b'x' b'_' b'_' b'_' b'_' b'x' b'_' b'_']]
Please input the actions (up: 1, down: 2, left: 3, right: 4): 1
step: 3, state: (2, 0, 1, 1), actions: 1, reward: -16
```

3. Robustness study under action noise. The provided environment is deterministic. Without modifying the map, reward function, or step() implementation, add action noise in the training/evaluation loop: with probability p, replace the selected action by one of the other three actions uniformly at random; otherwise execute the selected action. Train three agents using p_{train} in {0, 0.1, 0.2}. Then evaluate each trained policy under p_{test} in {0, 0.1, 0.2} for at least 100 episodes per setting. During evaluation, disable any additional exploration and keep only the specified action noise. For this study, provide: (i) episode reward curves for the three training settings; (ii) a 3x3 heatmap or table of success rate across all (p_{train}, p_{test}) pairs; and (iii) a 3x3 heatmap or table of average episode return across all pairs. Discuss how action noise affects learning and whether training with moderate noise improves robustness.

4. Deliverables

You can use the code to get the state and finish your own RL code as well as the report.

Submission due:

Please submit through NTULearn Website.

Submission files:

- The ipynb file that includes your code implementation. You need to add text block or comments to your implementation and visualize the learning progress: episode rewards vs. episodes, and the final V-table and policy.
- A pdf file that includes your report, which contains the description of your chosen algorithm and the plots such as learning progress, policy, etc. from the ipynb file.
- Your final submission should be compressed into a zip file.

The report filename format:

- AI6101_Report_YourName_YourMatriculationNumber.pdf

The code filename format:

- AI6101_Code_YourName_YourMatriculationNumber.ipynb

Please put the two files into a zip file with the filename format:

- AI6101_Project_YourName_YourMatriculationNumber.zip

5. Marking Criteria

The grading criterion are as follows (total 100 marks):

Item	Marks
Bug-free: correctly implement the code of your chosen RL algorithms and visualization	40%
Plot the learning progress: episode rewards vs. episodes	20%
Final policy visualization: clearly show the learned policy	20%
Action-noise robustness study: analyze training performance and cross-noise generalization using the train/test noise evaluation matrix	20%

A bonus of **10** mark is awarded for each of the following tasks:

- Compare different exploration techniques (such as UCB) with the default epsilon-greedy and non-exploration strategy. Come up with your analysis.
- Use only sparse reward (the agent only receives a reward of 1000 if the box reaches the goal position) and implement your ideas to solve this problem (hint: exploration technique, hierarchical RL, etc.).

Nevertheless, your final project mark will still be capped at 100% even if the total mark plus the bonus mark exceeds 100%. You can refer to option-critic as your HRL solution.